

**МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ
ФЕДЕРАЦИИ
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ БЮДЖЕТНОЕ ОБРАЗОВАТЕЛЬНОЕ
УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
«НОВОСИБИРСКИЙ ГОСУДАРСТВЕННЫЙ ТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ»**

КАФЕДРА ВЫЧИСЛИТЕЛЬНОЙ ТЕХНИКИ



**НГТУ
НЭТИ**

Лабораторная работа №3

по дисциплине «ТРПО-2»

на тему

**«Разработка мобильного приложения на Android,
взаимодействующего с классом структуры данных»**

Группа:

Факультет: АВТФ

Студенты:

Преподаватель: Романов Е.Л.

Новосибирск 2022

Оглавление

1	Задание на расчётно-графическую работу	3
2	Исходные данные	3
3	Ход работы	3
3.1	Типы данных	3
3.2	Паттерны/шаблоны проектирования	3
3.2.1.	Фабрика (Factory)	3
3.2.2.	Объект-прототип/Строитель (Prototype/Builder)	4
3.3	Интерфейсы	5
3.4	Структура бинарного дерева в массиве	5
3.4.1.	Вставка	6
3.4.2.	Поиск по индексу	7
3.4.3.	Удаление	8
3.4.4.	Балансировка	9
3.4.5.	Итератор	11
3.5	Разработка приложения на Android	11
3.5.1.	Сохранение(загрузка) в(из) файл(а)	11
3.5.2.	Элементы интерфейса	13
3.5.4.	Примеры обработки исключительных ситуаций	15
3.5.5.	Пример работы разработанного ПО	17
	Заключение	20
	Приложение А	20

1 Задание на расчётно-графическую работу

Портировать из Java разработанный в л.р.1 программный код в Android.

2 Исходные данные

Согласно варианту задания на лабораторную работу имеем:

- Структура данных: бинарное дерево на основе массива;
- Типы данных: Целочисленный, Дата-Время.

3 Ход работы

3.1 Типы данных

Целочисленный тип данных реализован в классе *IntegerClass*. Данный класс содержит в себе единственное поле – значение типа *int* (целочисленный тип). Реализованы геттеры и сеттеры, переопределён метод *toString()*.

Тип Дата-Время более комплексный и реализован в классе *DateTimeClass*. Содержит в себе поля: день, месяц, год, час, минута, секунда. Все значения полей класса - целочисленные. Реализованы проверки на установку настоящей даты (проверка на високосный год и количество дней в месяце, принадлежность значения заданному интервалу). Реализованы геттеры и сеттеры, переопределён метод *toString()*.

Код реализации типов данных представлен в Приложении А.

3.2 Паттерны/шаблоны проектирования

3.2.1. Фабрика (Factory)

В лабораторной работе №1 был реализован шаблон проектирования «Фабрика». Суть фабрики сводится к тому, чтобы производить объекты разного типа. В методе *getBuilderByName(name: String): ProtoType* фабрика получает на вход строку с названием выбранного типа данных, а возвращает объект-прототип.

```

    private final static ArrayList<ProtoType> typeNameList =
new ArrayList<>();

    static {
        ArrayList<ProtoType> buildersClasses = new
ArrayList<>(Arrays.asList(new DateTimeType(), new
IntegerType()));
        typeNameList.addAll(buildersClasses);
    }
    public static ArrayList<String> getTypeNameList() {
        ArrayList<String> typeNameListString = new
ArrayList<>();
        for (ProtoType protoType : typeNameList) {
            typeNameListString.add(protoType.typeName());
        }
        return typeNameListString;
    }
    public static ProtoType getBuilderByName(String name){
        if (name == null){
            throw new RuntimeException("Error! Name of type is
empty!");
        }
        for (ProtoType userType : typeNameList) {
            if (name.equals(userType.typeName()))
                return userType;
        }
        return null;
    }
}

```

Листинг 1. Реализация шаблона проектирования «Фабрика»

3.2.2. Объект-прототип/Строитель (Prototype/Builder)

В лабораторной работе №1 был реализован шаблон проектирования «Объект-прототип/Строитель». Суть его в том, чтобы обеспечить дополнительную обёртку для классов типов данных. Для реализации данного паттерна были переопределены для каждого типа данных все стандартные методы, а именно: *typeName()*, *create()*, *clone()*, *readValue()*, *parseValue()*, *getTypeComparator()*, *toString()*. Назначение каждого метода описано в комментариях к реализации шаблона проектирования (листинг 2). В конечном итоге были реализованы ещё два класса, наследующие интерфейс *ProtoType*: *IntegerType* и *DateTimeType*.

Код реализации этих классов приведён в Приложении А.

```
public interface ProtoType {
    // Имя типа
    public String typeName();
    // Создание объекта
    public Object create();
    // Клонирование текущего
    public Object clone(Object obj);
    // Создание и чтения объекта
    public Object readValue(InputStream inputStream);
    // Создает и парсит содержимое из строки
    public Object parseValue(String someString);
    // Возврат компаратора для сравнения
    public Comparator getTypeComparator();
}
```

Листинг 2. Интерфейс для шаблона «Объект-прототип/Строитель»

3.3 Интерфейсы

Помимо уже рассмотренного интерфейса *ProtoType*, был реализован интерфейс компаратора *Comparator* (листинг 3). Данный интерфейс содержит в себе только один метод для переопределения – *compare(o1: Any?, o2: Any?): Int*, который позволяет сравнить значения соответствующего типа данных. Были реализованы два класса наследующий данный интерфейс: *IntegerComparator* (используется для сравнения целочисленных значений) и *DateTimeComparator* (используется для сравнений значений типа данных Дата-Время). Код реализации этих классов приведён в Приложении А.

```
public interface Comparator {
    public int compare(Object o1, Object o2);
}
```

Листинг 3. Интерфейс для реализации компаратора

3.4 Структура бинарного дерева в массиве

Структура бинарного дерева была подробно рассмотрена в лабораторной работе №1.

Класс *BinaryTreeArray* – бинарного дерева в массиве содержит следующие поля:

- *ArrayList <Object> arrayTree* – исходный динамический массив, который хранит объекты – узлы дерева;
- *int size* – поле, обозначающее размер массива;
- *Comparator comparator* – компаратор, он предназначен для сравнения объектов внутри методов класса.

Для работы со структурой реализованы следующие методы:

- *void addValue(Object value)* – добавление объекта в дерево;
- *void printTree()* – печать дерева в консоль;
- *void printArray()* – печать массива в консоль;
- *Object getDataAtIndex(int searchIndex)* – получение значения по индексу;
- *void removeNodeByIndex(int index)* – удаление по индексу;
- *void forEach(DoWith func)* – итератор;
- *BinaryTreeArray balance()* – балансировка дерева;
- *String toString()* – преобразование дерева в строку (необходимо для GUI).

Код реализации класса *BinaryTreeArray* приведён в Приложении А.

Рассмотрим основные алгоритмы данной структуры.

3.4.1. Вставка

Включение нового значения в двоичное дерево также базируется на ветвлении: при сравнении включаемого значения со значением в текущей вершине выбирается правая или левая ветвь до тех пор, пока не встретится свободная (листинг 4). При этом, если индекс вставки будет больше, чем размер массива, то массив необходимо увеличить.

```
// Вспомогательный метод вставки значения в массив
private void insertRecursive(int current, Object obj){
    while (current >= size) {
        size *= 2;
```

```

        for (int i = size / 2; i <= size; i++) // с
обнулением новой части
            arrayTree.add(null);
    }
    if (arrayTree.get(current) == null) {
        arrayTree.set(current, obj);
        return;
    }
    if (comparator.compare(obj, arrayTree.get(current)) <
0)
        insertRecursive(2 * current + 1, obj);
    else
        insertRecursive(2 * current + 2, obj);
}
// Вставка значения в дерево
public void addValue(Object value) {
    insertRecursive(0, value);
}

```

Листинг 4. Метод вставки нового значения в дерево

3.4.2. Поиск по индексу

Поиск заданного значения в двоичном дереве использует линейно рекурсивный алгоритм. В каждой вершине производится сравнение размера левого поддерева с искомым номером. Если искомый номер меньше, чем размер левого поддерева, то рекурсивно ищется в левом поддереве. Если нет – то от искомого индекса вычитаются число вершин левого поддерева. Далее если декремент получившейся величины совпадает с нулём, то элемент найден, а если нет, то ищется индекс в правом поддереве.

Реализована индексация вершин по логическому номеру начиная с нуля «слева-направо» (листинг 5).

```

// Число вершин в поддереве
private int getSize(int num) {
    if (num >= size || arrayTree.get(num) == null)
        return 0;
    return 1 + getSize(2 * num + 1) + getSize(2 * num +
2);
}

```

```

    private Object getDataAtIndexRecursive(int searchIndex,
int help){
        if (searchIndex >= size || searchIndex >=
getSize(help))
            return null;

        int cntL = getSize(2 * help + 1); // число вершин в
левом поддереве

        if (searchIndex < cntL)
            return getDataAtIndexRecursive(searchIndex, 2 *
help + 1); // Логический номер в левом поддереве

        searchIndex -= cntL; // отбросить вершины левого
поддерева

        if (searchIndex-- == 0)
            return arrayTree.get(help); // Логический номер -
номер текущей вершины

        return getDataAtIndexRecursive(searchIndex, 2 * help +
2); // в правое поддерево с остатком Логического номера
    }

    //нумерация "слева-направо", начинается с 0, см. cprog 8.5
    public Object getDataAtIndex(int searchIndex){
        return getDataAtIndexRecursive(searchIndex, 0);
    }

```

Листинг 5. Метод поиска значения в дереве

3.4.3. Удаление

В первую очередь происходит поиск удаляемого элемента, после нахождения которого следует этап, у которого могут быть три вариации:

- 1 *Удаляемый узел является листовым (не имеет потомков).*

В данном случае просто удаляем лист из дерева.

- 2 *Удаляемый узел имеет одного потомка.*

В таком случае мы удаляем выбранный узел, и на его место ставим его потомка. После это сдвигаем поддерева переставляемого узла в массиве так, чтобы сохранились связи.

- 3 *Удаляемый узел имеет двух потомков.*

В данном случае, необходимо в левом поддереве найти максимальный элемент (он же будет листом) и переставить его на место удаляемого узла.

Так как реализация удаления из дерева вышла достаточно объёмная, то ознакомиться с ней можно в Приложении А с подробными комментариями.

3.4.4. Балансировка

Реализована рекурсивная балансировка на языке программирования *Kotlin* – выполняется обход с сохранением упорядоченной последовательности массива, при котором индексы узлов собираются в *Vector*. Затем *Vector* рекурсивно делится пополам, значение из середины включается в новое дерево. Итоговое дерево получается сбалансированным (*cprog*, раздел 8.5). Метод возвращает новый объект *BinaryTreeArray* (листинг 6).

```
//рекурсивная балансировка
private void balance(Vector<Object> t, int a, int b,
ArrayList<Object> r) {

    if (a>b) return;
    if (a==b) return;

    int m=(a+b) >>> 1; // взять строку из середины
интервала

    insertRecursive(r, 0,t.get(m));

    balance(t, m+1,b, r); //рекурсивно выполнить для левой

    balance(t, a,m,r); // и правой частей

}

//вставка для нового аппрайлист при балансировке
private void insertRecursive(ArrayList<Object> t, int
current, Object obj){
    if (current >= size){ // увеличение размерности при
выходе
        size *= 2; // за пределы массива
        for (int i = size/2; i <= size; i++) // с
обнулением новой части
```

```

        t.add(null);
    }

    if (t.get(current) == null) {
        t.set(current, obj);
        return;
    }

    if (comparator.compare(obj, t.get(current)) < 0)
        insertRecursive(t, 2 * current + 1, obj);
    else
        insertRecursive(t, 2 * current + 2, obj);
}

//главный метод балансировки
public BinaryTreeArray balance(){

    int sz1=getSize(0);

    Vector <Object> newArray = new Vector<Object> (size);
//вектор индексов

    ArrayList<Object> newArrayTree = new
ArrayList<Object>(size);
    for(int i = 0; i < size; i++) {
        newArrayTree.add(null);
    }
    set(newArray,0);
    balance(newArray,0, sz1, newArrayTree);
    BinaryTreeArray balanced = new
BinaryTreeArray(this.size, newArrayTree, this.comparator);
    return balanced;

}

//метод для добавления индексов в вектор
private void set(Vector<Object> t, int n){

    if (n>=size || arrayTree.get(n) == null) return;

    set(t,2*n+1);
    t.add(arrayTree.get(n));
    set(t,2*n+2);

}

```

Листинг 6. Балансировка дерева

3.4.5. Итератор

Под итератором *forEach* понимается метод (листинг 7), который проходит структуру данных линейно и вызывает переданную лямбду (листинг 8) для каждого хранимого элемента данных, а если лямбда срабатывает, то возвращает.

```
// итератор forEach
public void forEach(DoWith func)
{
    if (arrayTree == null || size <= 0)
        return;
    int sz = getSize(0);
    Vector <Integer> v = new Vector<Integer>(size);
    setHelp(v, 0);
    for(int i=0; i < sz; i++)
    {
        func.doWith(arrayTree.get(v.get(i)));
    }
}
```

Листинг 7. Реализация итератора

```
public interface DoWith {
    public void doWith(Object obj);
}
```

Листинг 8. Интерфейс для реализации функционально тривиальных операций над элементами списка структуры данных

3.5 Разработка приложения на Android

3.5.1. Сохранение(загрузка) в(из) файл(а)

В классе мобильного приложения имеются методы сохранения дерева в файл и загрузки его из файла (листинг 9). Сериализация и десериализация представляет собой сохранение и загрузку хранимых объектов дерева, а не самого дерева.

```
public void loadBinaryTreeArray(View view) throws
IOException {
    BufferedReader br;
    try {
        br = new BufferedReader(new
InputStreamReader((openFileInput("saved.txt"))));
    }
    catch (Exception ex) {
```

```

        Toast.makeText(getBaseContext(), "Файл не
существует!", Toast.LENGTH_LONG).show();
        return;
    }
    String line;
    line = br.readLine();
    if (line == null) {
        Toast.makeText(getBaseContext(), "Файл пуст!",
Toast.LENGTH_LONG).show();
        return;
    }
    if (!protoType.typeName().equals(line)) {
        generateAlertDialog("Ошибка!",
            "Измените тип данных, чтобы загрузить
файл!",
            "Измените тип!");
        return;
    }
    btsArray = new
BinaryTreeArray(protoType.getTypeComparator());

    while ((line = br.readLine()) != null) {
        try {
            btsArray.addValue(protoType.parseValue(line));
        }
        catch (Exception ex) {
            generateAlertDialog("Ошибка при чтении файла",
"В нем хранится другая структура данных!", "Повторите
действие");
            br.close();
            return;
        }
    }
    br.close();

    Toast.makeText(getBaseContext(), "Файл загружен",
Toast.LENGTH_LONG).show();
    rewrite();
}

public void saveBinaryTreeArray(View view) throws
IOException {
    BufferedWriter br = new BufferedWriter(new
OutputStreamWriter((openFileOutput("saved.txt",
MODE_PRIVATE))));
    br.write(protoType.typeName() + "\n");
    btsArray.forEach(el -> {
        try {
            br.write(el.toString() + "\n");
        } catch (IOException e) {
            e.printStackTrace();

```

```

    }
    });
    br.close();

    Toast.makeText(getApplicationContext(), "Файл сохранён",
    Toast.LENGTH_LONG).show();
}

```

Листинг 9. Реализация сохранения в файл и загрузки из файла

3.5.2. Элементы интерфейса

На рис. 1 представлен графический интерфейс мобильного приложения на Android. Разберём его элементы:

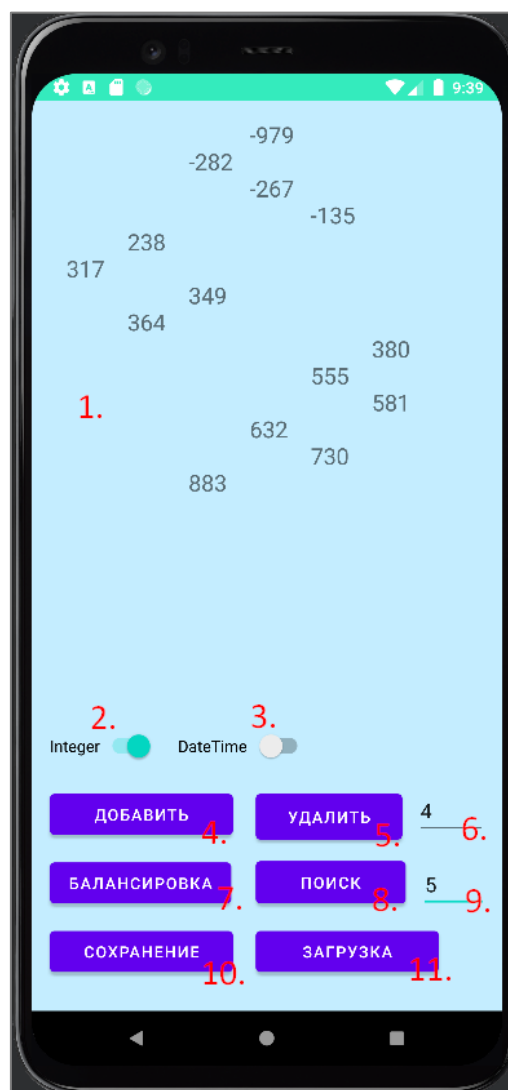


Рисунок 1. GUI мобильного приложения

- Под номером 1 – текстовое поле типа *TextView*, в котором отображается структура дерева. При выходе за границы

данного поля можно прокрутить его, чтобы увидеть вышедшие за его границы значения.

- Под номерами 2 и 3 – слайдеры, отвечающие за смену типа данных дерева. При смене типа текстовое поле очищается. Слайдеры не могут работать оба во включенном режиме.
- Под номером 4 – кнопка добавления элемента со случайным значением в дерево;
- Под номерами 5 и 6 – кнопка для удаления элемента из дерева по индексу и редактируемое текстовое поле для ввода индекса удаляемого элемента соответственно;
- Под номером 7 – кнопка балансировки текущего дерева;
- Под номерами 8 и 9 – кнопка для поиска элемента в дереве по индексу и редактируемое текстовое поле для ввода индекса искомого элемента соответственно;
- Под номерами 10 и 11 – кнопки сохранения объектов дерева в файл и загрузки объектов дерева из файла соответственно.

Также при смене ориентации экрана мобильного устройства можно пользоваться разработанным приложением (рис. 2).

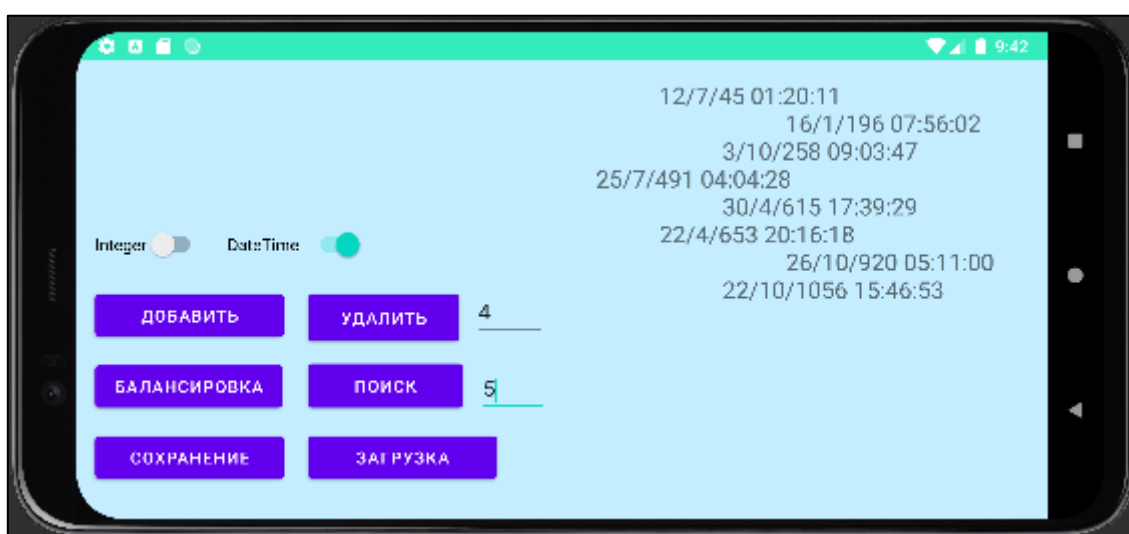


Рисунок 2. Смена ориентации мобильного устройства

3.5.4. Примеры обработки исключительных ситуаций

Также в работе приложения предусмотрена обработка таких исключительных ситуаций (вывод диалогового окна с соответствующей информацией) как:

- В поле ввода индекса для удаления сам индекс отсутствует (рис. 3). Для поиска значения в дереве поведение аналогично;

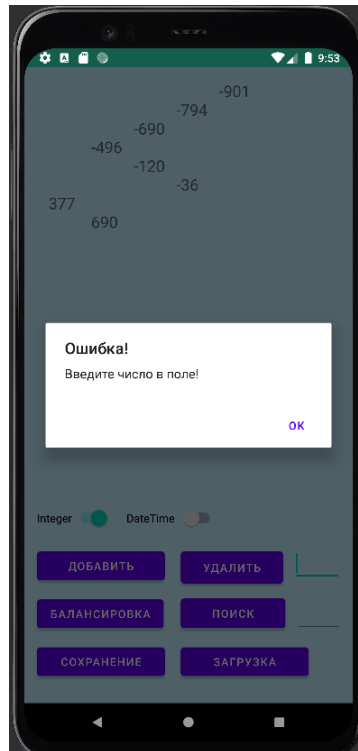


Рисунок 3. Исключительная ситуация

- В поле ввода индекса для удаления введен индекс, который не существует в текущем дереве. Для поиска значения в дереве поведение аналогично (рис. 4);

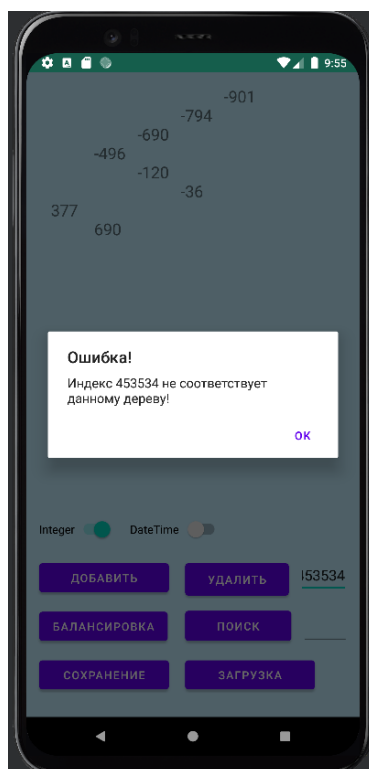


Рисунок 4. Исключительная ситуация

- Загружаемое дерево из файла не соответствует типу данных, которое выставлено в приложении (рис. 5);

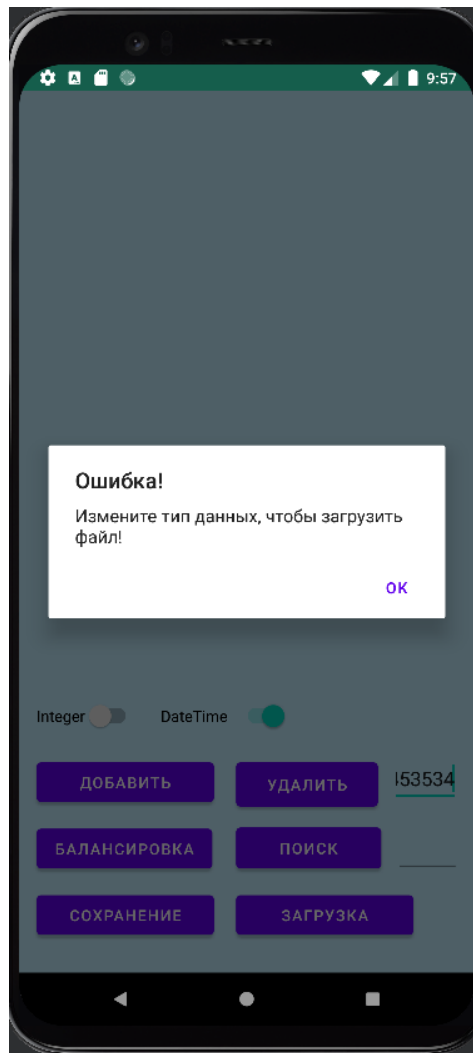


Рисунок 5. Исключительная ситуация

- Также присутствует обработка исключительных ситуаций таких, как загрузка несуществующего файла, ошибки чтения файла и т.д.

3.5.5. Пример работы разработанного ПО

На рис. 6 – 10 приведены примеры работы разработанного мобильного приложения.

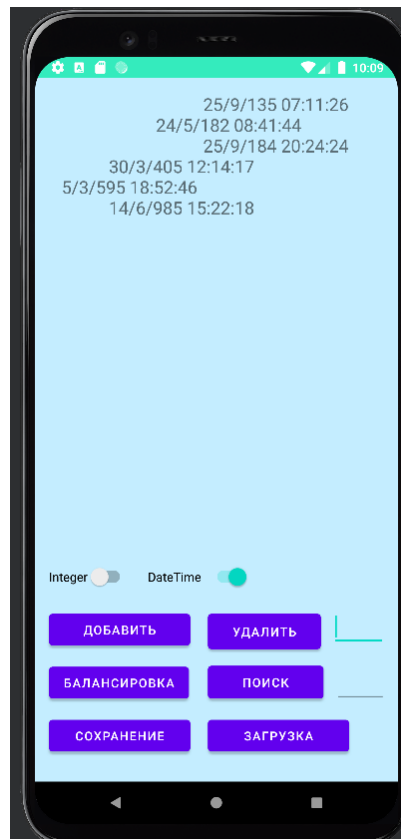


Рисунок 6. Добавление элементов в дерево

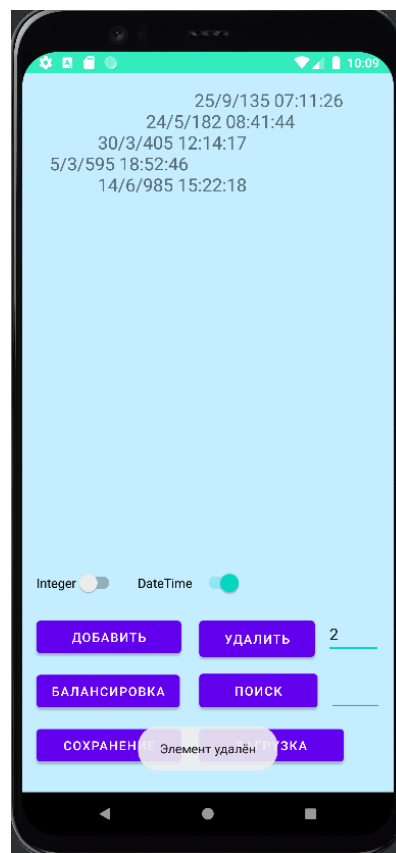


Рисунок 7. Удаление элемента из дерева по индексу 2

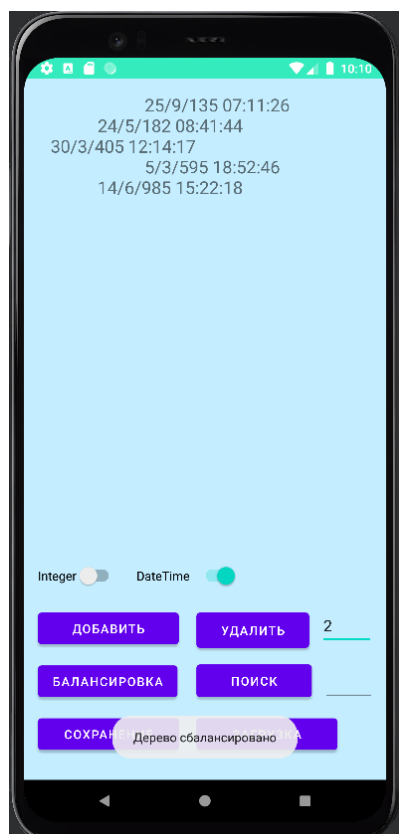


Рисунок 8. Сохранение текущего состояния дерева в файл и балансировка

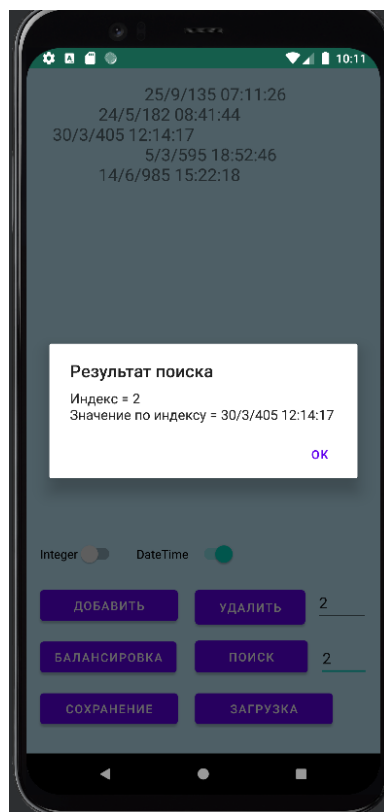


Рисунок 9. Поиск элемента по индексу 2

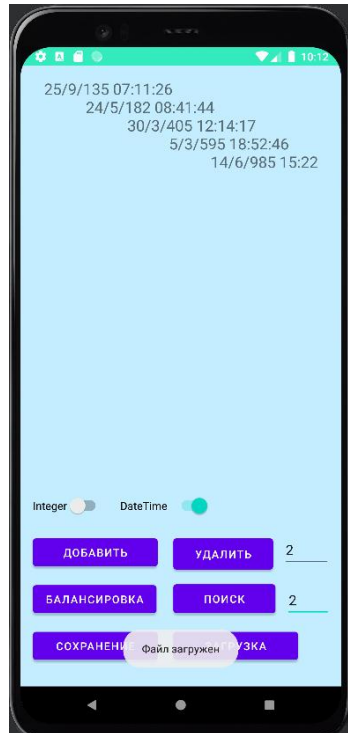


Рисунок 10. Загрузка объектов дерева из файла

Заключение

В ходе выполнения расчётно-графической работы было разработано и протестировано программное обеспечение в виде мобильного приложения на Android, позволяющее работать со структурой данных «Бинарное дерево в массиве» с различными типами данных (целочисленный тип и дата-время). Были применены шаблоны/паттерны проектирования «Фабрика» и «Объект-прототип/Строитель».

Программный код данного ПО был портирован в *Android*. Был сделан вывод, что возможности *Android* для разработки мобильных приложений являются достаточно удобными.

Приложение А

Весь исходный код расположен по ссылке:

Арк-файл мобильного приложения: